Zealand

Fag: **Udvidet netværksteknologi -** Sune Andersen (SUAN).

Opgave 6 – Cæsar-kryptering, kryptanalyse og AI

Formål

I denne opgave skal du undersøge, hvordan en simpel kryptering kan brydes. Du skal programmere et brute-force-angreb i Python og derefter undersøge, hvordan AI kan hjælpe med at vælge den mest sandsynlige klartekst. Til sidst skal du vurdere, hvornår AI's forslag er til at stole på.

Kort teori

Klartekst er den oprindelige, læsbare besked. Kryptotekst er den krypterede besked.
Ved Cæsar-kryptering flyttes hvert bogstav et fast antal pladser i alfabetet. Nøglen fortæller, hvor mange pladser der flyttes. Ved dekryptering flyttes bogstaverne samme antal pladser tilbage. Mellemrum bevares.

Eksempel:
Med nøgle 3 bliver A til D ved kryptering. Ved dekryptering flyttes D tre pladser tilbage til A.
Der er kun 26 mulige forskydninger i dette alfabet, så en computer kan afprøve dem alle.



Del 1 – Bryd krypteringen med brute force (øjet ser den rigtige på skærmen)

Du har fået denne kryptotekst. Du kender ikke nøglen: **`FBEHUVHFXULWB SDD CHDODQG`**

1. Gennemgå og skriv Python-programmet nedenfor færdig.
2. Kør programmet, og find den nøgle, der giver en meningsfuld tekst.
3. Skriv klarteksten ud på skærmen

Bemærk : Det engelske sprog er 26 tegn (Mangler Danske tegn)

```
# Eng. version.... Uden danske tegn...
alfabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
kryptotekst = "Indsæt krypto teksten her"
for noegle in range(xx):
    klartekst = ""
    for bogstav in kryptotekst:
        if bogstav in alfabet:
            position = alfabet.index(bogstav)
            ny_position = (position - noegle) % xx
            klartekst += alfabet[ny_position]
        else:
            klartekst += bogstav
    print(noegle, klartekst)
```

Del 2 – Undersøg AI

Kopiér resultater fra del 1 ind i et AI-værktøj, som du daglig bruger.
Bed AI om at rangere kandidaterne efter, hvor sandsynligt det er, at de er en tilsigtet besked.

Teksten du kan bruge til at spørge ChatGPT eller den AI du bruger.

```
Du hjælper med kryptanalyse i en undervisningsopgave.
Her er muligerne dekrypteringer af en Cæsar-kryptering:
[indsæt alle resultater her]
Rangér de tre mest sandsynlige klartekster.
Forklar kort, hvilke sproglige tegn du bruger.
Angiv den tilhørende nøgle, hvis den fremgår af listen.
Vær tydelig om usikkerhed. Opfind ikke en tekst, der ikke står på listen.
```

Sammenlign AI's valg med dit eget valg og med resultatet fra del 1.
AI skal ikke automatisk betragtes som facit: den kan gætte forkert, og
en kort tekst giver kun lidt information.

Ekstra opgave: Skriv det nu ind i dit Python program – så dit program automatisk kan udvælge den rigtige sætning. -> HUSK din AI-key!

Spørgsmål til besvarelsen (SKAL ikke afleveres)

1. Hvad er klarteksten, kryptoteksten og nøglen i denne opgave?
2. Hvorfor er Cæsar-kryptering let at bryde med brute force?
3. Hvilken kandidat valgte din egen vurdering, og hvilken valgte AI? Var de enige?
4. Hvilke tegn brugte AI til at vælge? Var forklaringen overbevisende?
5. Kan man være sikker på, at den mest meningsfulde kandidat er den oprindelige besked? Hvorfor/hvorfor ikke?
6. Hvorfor bør Cæsar-kryptering ikke bruges til at beskytte fortrolige data i moderne IT-systemer?

Kunne man tilføje noget så teksten bliver svære at cracke? (kig på SALT!)

Caesar: Kan knækkes ved at prøve alle forskydninger.
SHA-256 uden salt: Hurtig at beregne og derfor uegnet alene til lagring af fortrolige data.
Scrypt med tilfældigt salt: Gør hvert gæt dyrere og forhindrer direkte genbrug af de samme forudberegnede hashværdier på tværs af forskellige salte.

For at lege med SALT skal du bruge:
import os
import hashlib
import hmac
import base64

TIP : Undgå fejl på Ubuntu Linux (WSL / SSH):

Hvis I vil bruge OpenAI-modeller, er den centrale import Python: from openai import OpenAI

Ubuntu (bash som root) :

```
python3 -m venv .venv
source .venv/bin/activate
sudo apt update -y
sudo apt install -y python3-full python3-venv
python -m pip install openai
python -c "from openai import OpenAI; print('OpenAI-pakken er installeret')"
```